

El bug silencioso era mi arreglo.

Open-Meteo devuelve las horas sin desfase y, en otro campo, el desfase que ha aplicado. Ese campo parecía estar mal. La conclusión fue ignorarlo — y esa conclusión desplazaba una hora todas las temperaturas de invierno, sin que fallara nada.

El sesgo de una hora

La misma medianoche del 1 de marzo, vista por las dos fuentes y por los dos arreglos.



REE entrega la marca con su desfase real y es inequívoca. Open-Meteo no, y de ahí sale todo: reinterpretar su etiqueta como hora de Madrid empareja cada temperatura con la demanda de la hora siguiente.

La API no se equivocaba

Es coherente consigo misma; el error estaba en la lectura

Consultada en septiembre, Open-Meteo declara `utc_offset_seconds: 7200` para el 1 de marzo. Madrid ese día está en UTC+1, así que el campo parece un fallo de la API.

No lo es. **La API construye la serie aplicando el desfase que declara.** Que ese desfase sea el vigente el día de la petición y no el del día de los datos hace que las etiquetas no sean hora de Madrid — pero `etiqueta - utc_offset_seconds` da el UTC correcto siempre.

Comprobado pidiendo el mismo día dos veces, en `Europe/Madrid` y en `UTC`, y contrastando contra la segunda:

DÍA	DESFASE DECLARADO	REINTERPRETANDO A MADRID	RESTANDO EL CAMPO
2024-03-01	7200	21 de 24 mal	0 de 24
2024-01-15	7200	19 de 24 mal	0 de 24
2024-07-15	7200	0 de 24	0 de 24

En julio coincide porque Madrid sí está en UTC+2. **El error solo aparece en invierno**, que es justo la mitad del año en la que la temperatura más explica la demanda.

La solución no fue corregir el desfase aguas abajo, fue no tenerlo: la ingesta pide `timezone=UTC` y la marca ya es un instante.

Las decisiones de esta fase

01 Los datos se piden en UTC

Sin huso que deducir aguas abajo. Como el día local de Madrid empieza a las 22:00 o 23:00 UTC de la víspera, la petición cubre dos días UTC y la transformación recorta la ventana.

02 El parseo rechaza lo que no venga en UTC

Si `utc_offset_seconds` no es cero, el job revienta. El problema nunca fue equivocarse de huso: fue que equivocarse no producía ningún síntoma.

03

Una sola traducción entre husos

`ventana_utc(dia)` devuelve el intervalo UTC del día local. Todo lo demás trabaja en UTC. De paso explica sola el día de 23 y 25 horas: es la longitud de la ventana, no un caso especial.

04

Desfase explícito, no zona de sesión

La marca se parsea con un formato que lleva desfase, igual que la de REE. `to_timestamp` sobre una marca desnuda depende de `spark.sql.session.timeZone`, y esa es la clase de dependencia invisible que ya costó una hora.

05

El cruce conserva los huecos

Un `full outer`, no un `inner`. Si falta una hora en una fuente, la fila aparece con un nulo. Un `inner` habría dado una tabla más limpia y una mentira.

06

Proyección de particiones

Ni crawler ni `MSCK REPAIR`. Athena deduce las particiones del patrón de la ruta: coste cero y nada que mantener. Obliga a rutas predecibles, de ahí `mes=03` y no `mes=3`.

Por qué 28 tests en verde no lo vieron

No fue mala suerte. La suite estaba construida de forma que no podía verlo.

Comparar tu código contra tu código solo demuestra que eres consistente

La implementación de referencia en Python y la de PySpark aplicaban **la misma regla**. Si la regla es falsa, las dos fallan igual y el test sale verde. Había incluso un test llamado `test_temperatura_usa_las_reglas_horarias_y_no_el_desfase_declarado` , que codificaba la premisa equivocada y la protegía.

Ninguna comprobación contrastaba la salida contra la fuente.

Lo que lo destapó no fue un test: fue mirar la tabla. La temperatura mínima salía a las **09:00**, más de una hora después del amanecer del 1 de marzo. Con el arreglo cae a las **08:00**, que es donde tiene que estar.

El test que faltaba ya existe:

`test_la_temperatura_no_va_corrida_respecto_a_la_fuente` lee el JSON crudo y comprueba hora a hora que la temperatura que acaba en *curated* es la que la fuente pone en ese instante. Se ha verificado que **falla** si se reintroduce la conversión anterior.

31

TESTS EN TOTAL

26

SIN SPARK, EN MILISEGUNDOS

5

CON SESIÓN DE SPARK REAL

Regla que queda para el resto del proyecto: **al menos una comprobación por fuente tiene que atarse al dato de origen**, no a otra implementación propia.

El primer día curado

1 de marzo de 2024, 24 filas, cero nulos, cero errores y cero avisos.

Horas seleccionadas, en hora local de Madrid.

HORA LOCAL	DEMANDA (MW)	PVPC (€/MWH)	SPOT (€/MWH)	TEMP. (°C)
08:00	32.281	72,40	3,25	2,1
10:00	31.104	117,96	0,00	5,5
13:00	29.771	115,92	0,00	11,3
16:00	28.758	68,03	0,00	12,2
20:00	33.787	127,87	10,10	8,4

La curva que confirma que el desfase está bien

08:00

MÍNIMA, 2,1 °C

16:00

MÁXIMA, 12,2 °C

20:00

PICO DE DEMANDA, 33,8
GW

La mínima al amanecer y la máxima a media tarde. Si el desfase estuviera mal, la curva saldría corrida y se vería a simple vista — que es exactamente como se detectó el error.

Y ya asoma la pregunta del proyecto: de **10:00 a 16:00 el precio spot está a 0,00 €/MWh** mientras el PVPC va por encima de 115. La demanda hace pico a las 20:00, cuando el spot ya ha vuelto a subir. Marzo de 2024 fue de viento récord en España: la relación entre temperatura, consumo y precio no va a ser una recta.

Lo que sigue sin probarse

El job no se ha ejecutado en Glue. Queda por confirmar lo que solo existe en AWS: que el envoltorio de `awsglue` arranca, que el rol lee de `raw` y escribe en `curated`, y que Athena encuentra las particiones por proyección. Antes hay que reingestar el 1 de marzo, porque el JSON que hay en `raw` está en hora local y el parseo nuevo lo rechaza. La ejecución cuesta unos cuatro céntimos.

Caudalit · Fase 5 de 10 · Transformación

Siguiente: Athena — las queries que responden por fin la pregunta del proyecto